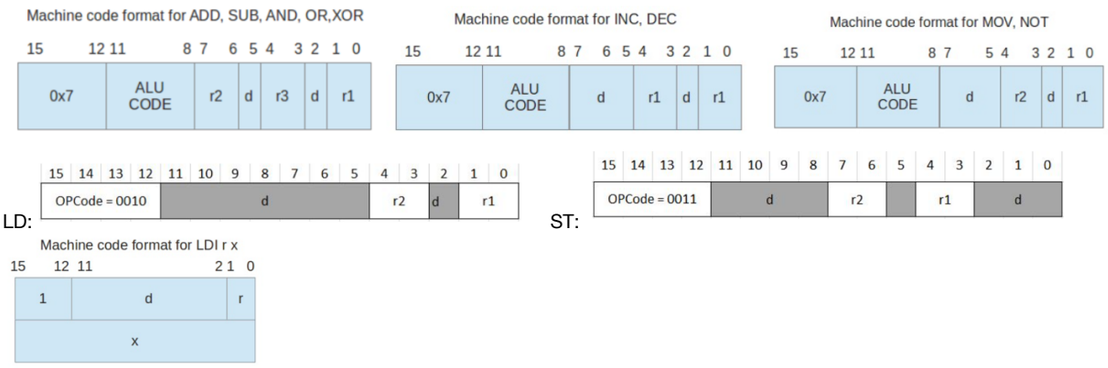
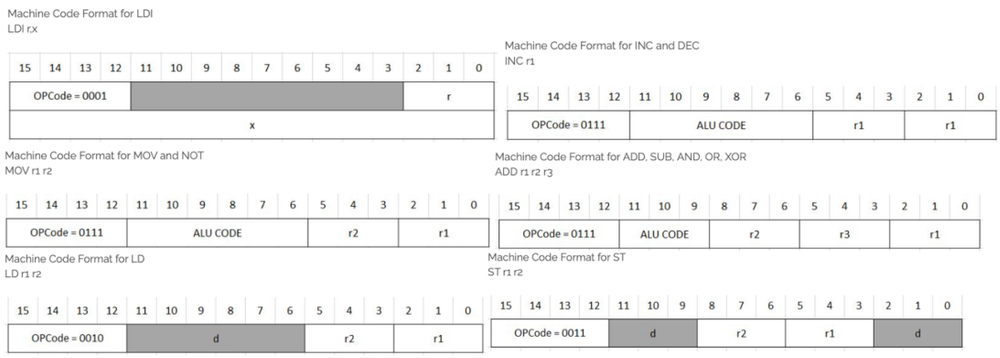
1. BASIC INSTRUCTIONS

ADD r1 r2 r3	mean	r1=r2+r3
SUB r1 r2 r3	mean	r1=r2-r3
NOT r1 r2	mean	r1=!r2
MOV r1 r2	mean	r1=r2
LD r1 r2	mean	r1=Memory[r2]
ST r1 r2	mean	Memory[r1]=r2
LDI r x	mean	r=x
INC r1	mean	r1++
DEC r1	mean	r1—

1.2. WHEN NUMBER OF REGISTERS IS 4:



1.3. WHEN NUMBER OF REGISTERS IS 8:



2. EVOLUTION HISTORY

2.1. KEY NOTES

Invertebrate
We only have a register bank and a ALU unit.

Fish
Instructions are now read from Memory. But still, we do not have ST and LD.

Frog
We now have LDI. But still, we do not have ST and LD.

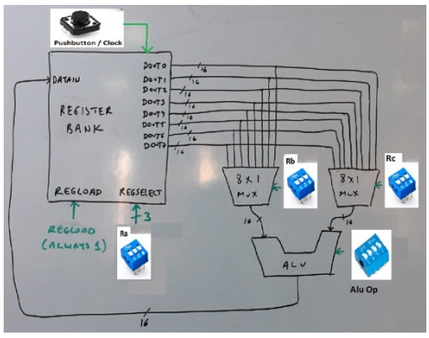
Reptile
We finally have ST and LD. Also, JMP and JZ instructions are introduced.

Bird
Stack instructions PUSH, POP, CALL, RET are introduced.

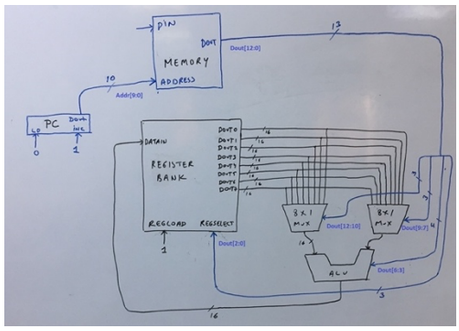
Vertebrate or Mammal (as our professor says)
Interrupt processing is added.

2.2. EVOLUTION OF CPU DESIGN

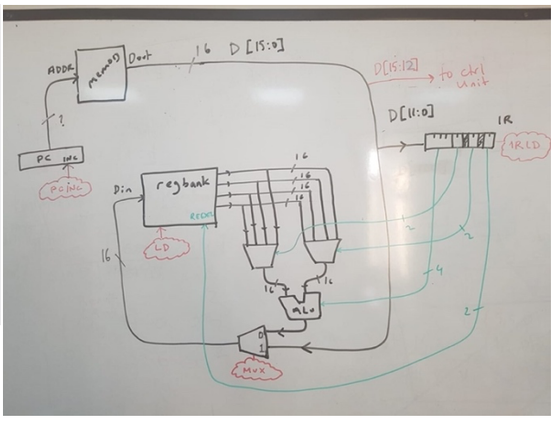
Invertebrate



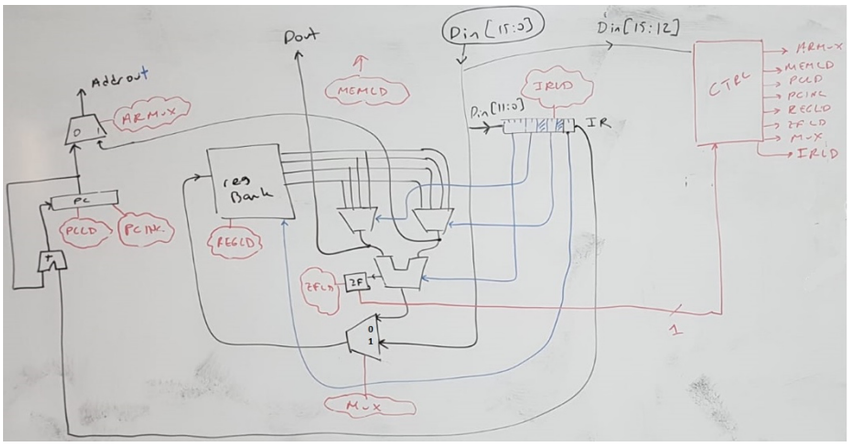
Fish



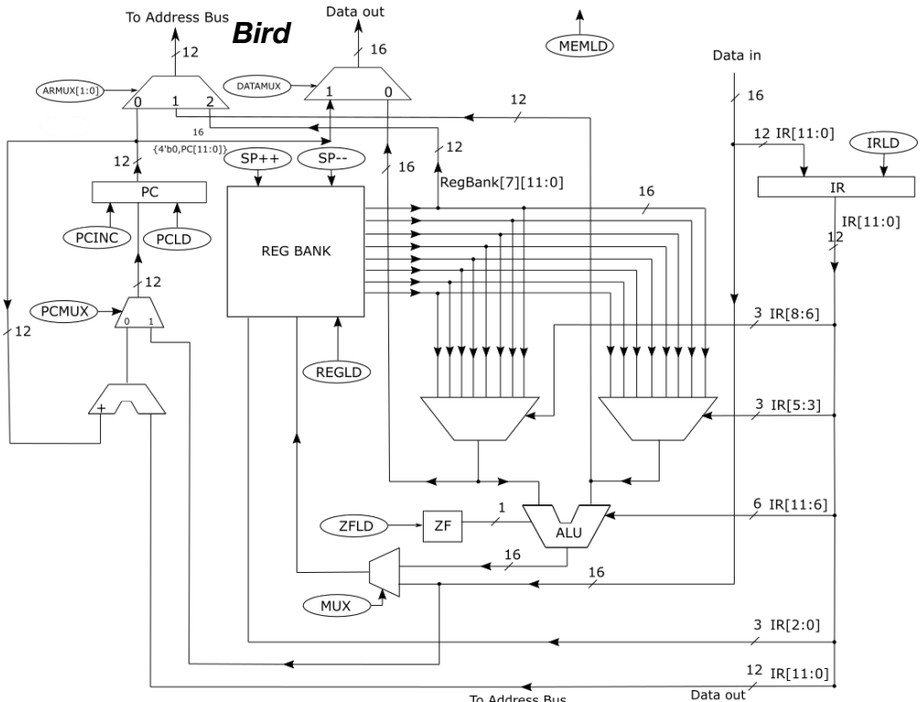
Frog



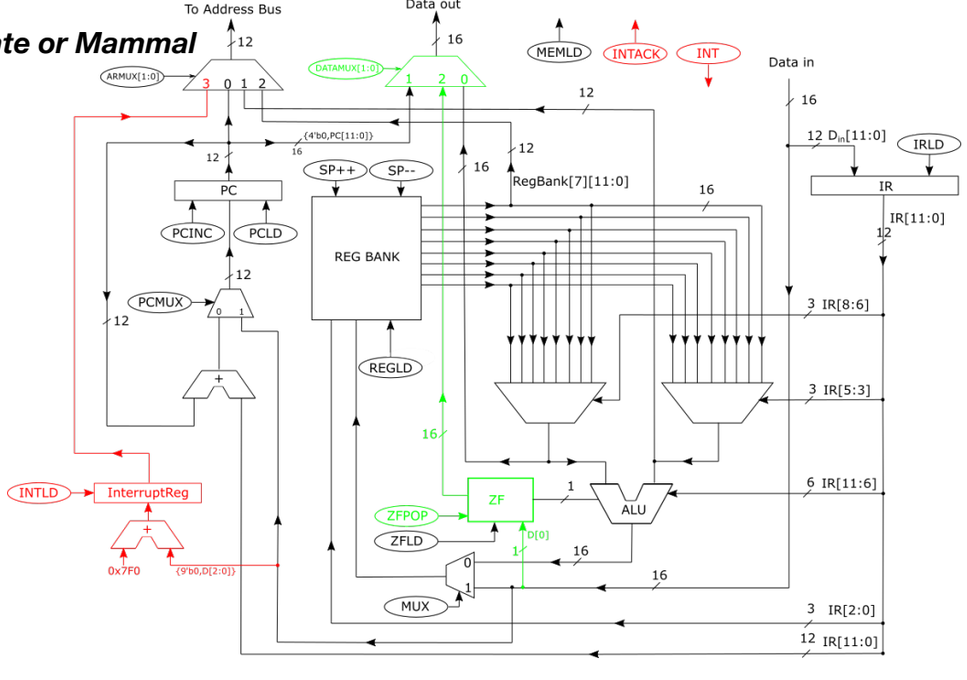
Reptile



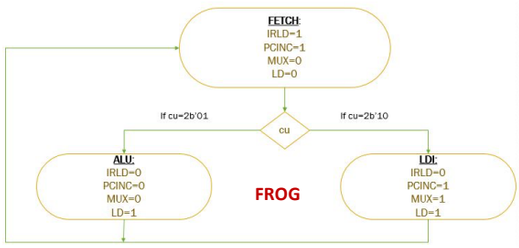
Bird

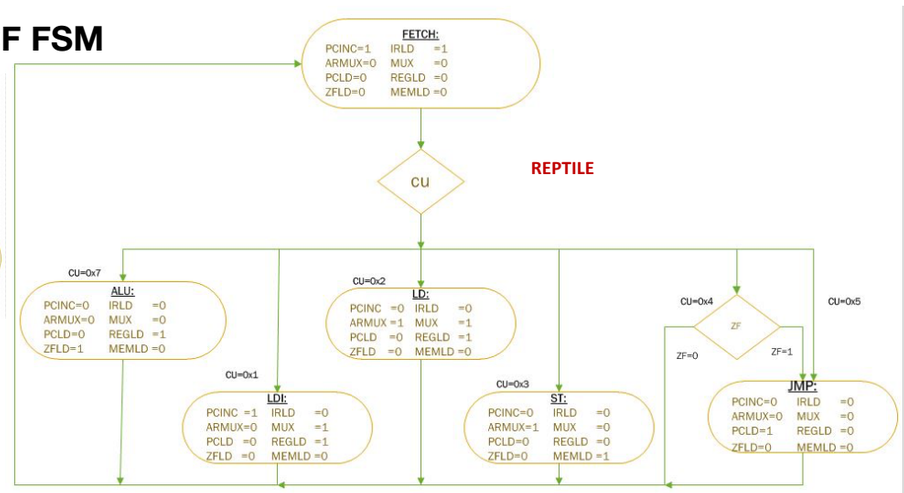


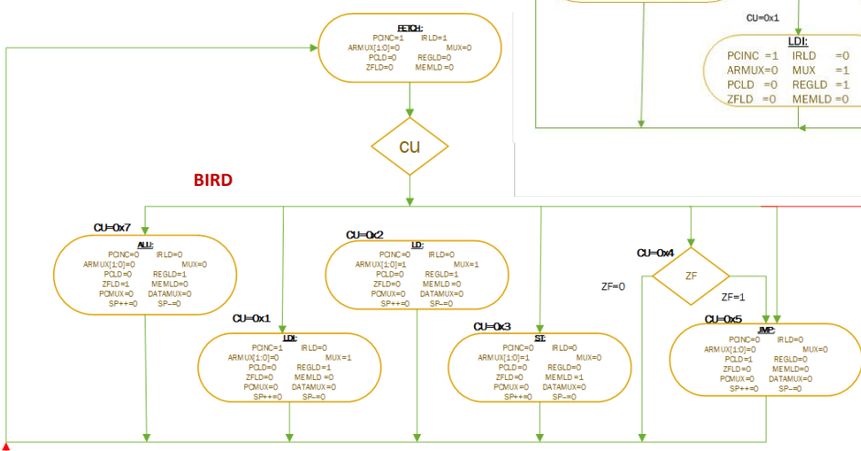
Vertebrate or Mammal

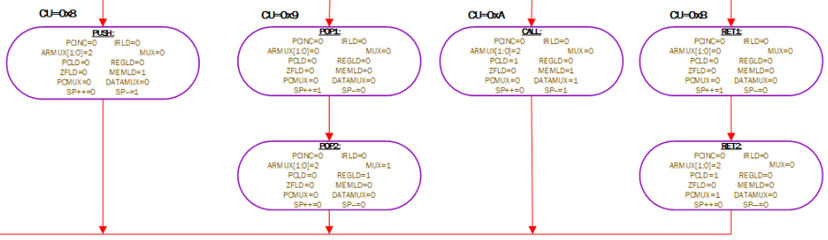


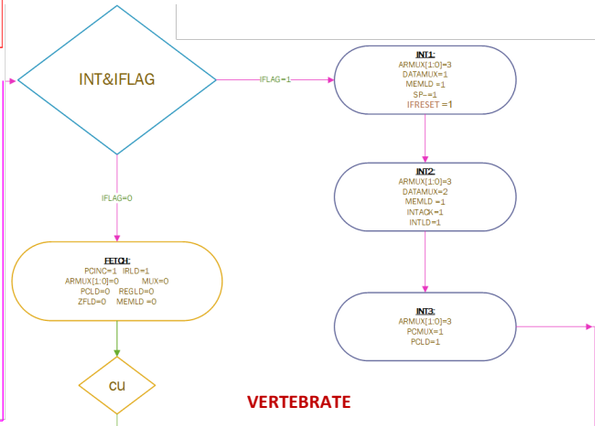
2.3. EVOLUTION OF FSM

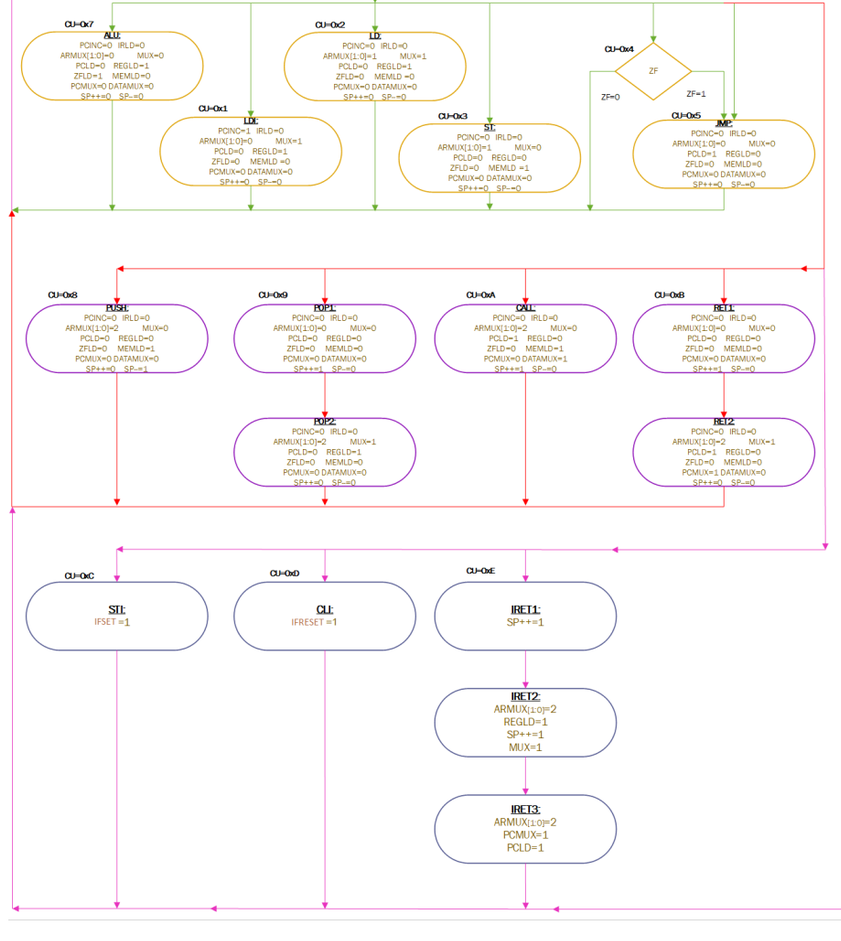




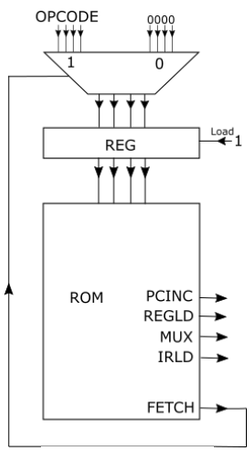




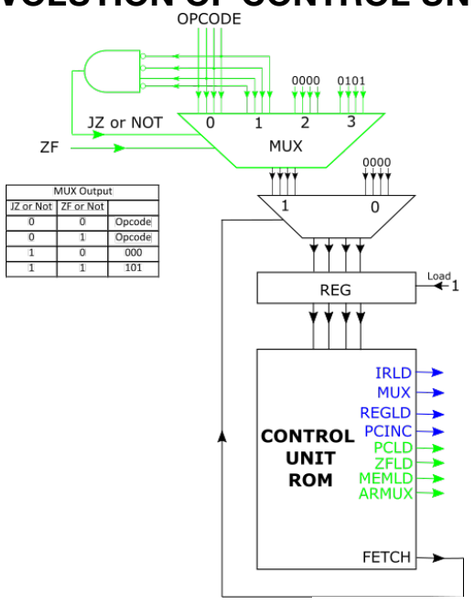




2.4. EVOLUTION OF CONTROL UNIT

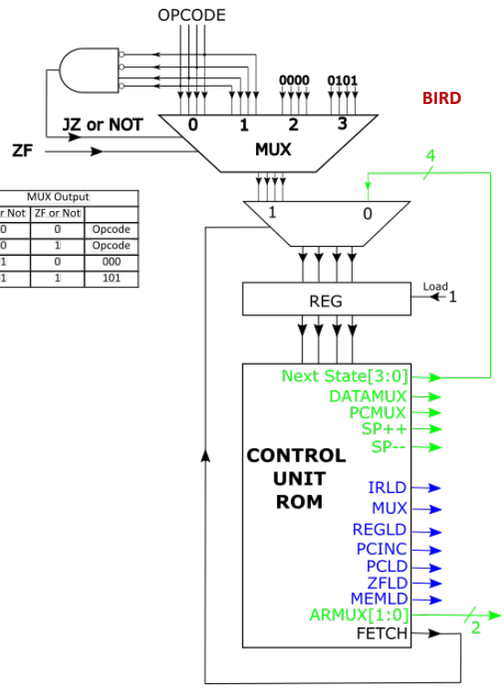


FROG

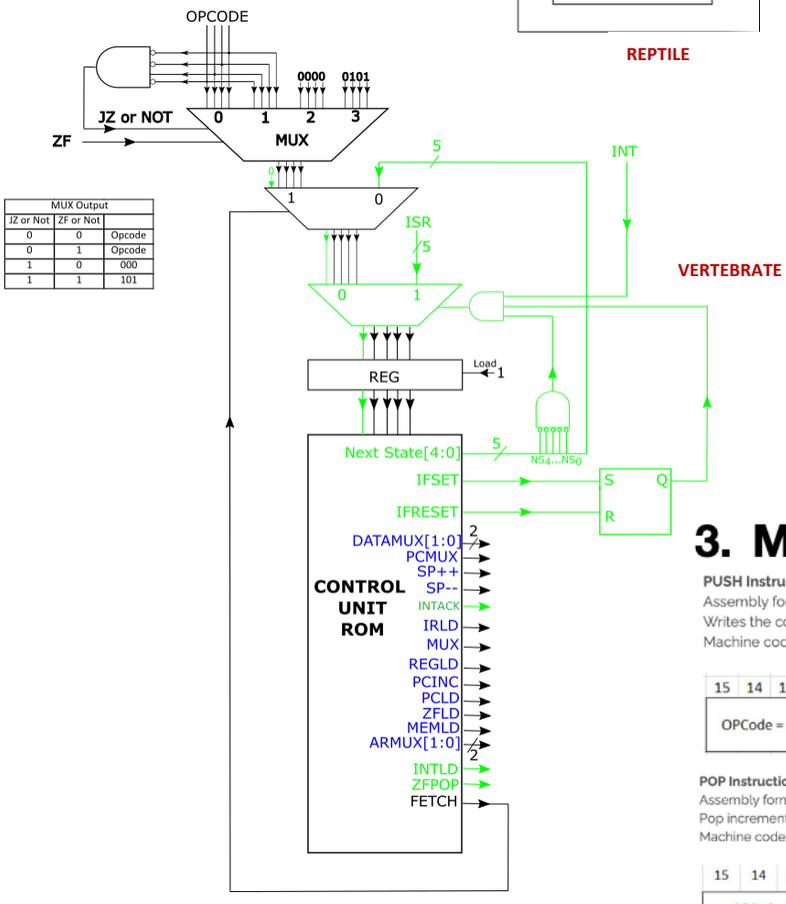


MUX Output		
JZ or Not	ZF or Not	
0	0	Opcode
0	1	Opcode
1	0	000
1	1	101

REPTILE



MUX Output		
JZ or Not	ZF or Not	
0	0	Opcode
0	1	Opcode
1	0	000
1	1	101



MUX Output		
JZ or Not	ZF or Not	
0	0	Opcode
0	1	Opcode
1	0	000
1	1	101

VERTEBRATE

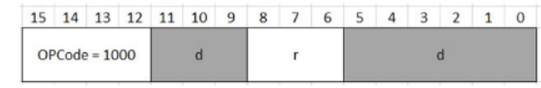
3. MORE INSTRUCTIONS

PUSH Instruction

Assembly format: Push r

Writes the contents of the register r into the memory location pointed by the SP. Then decrements SP by one.

Machine code format:

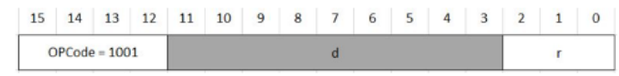


POP Instruction

Assembly format: Pop r

Pop increments SP by 1, then transfers the content of the memory location pointed by SP into register r.

Machine code format:



CALL Instruction

Assembly format: Call x

Pushes the content of the Program Counter (PC) into the stack and jumps to memory address PC+x, i.e, loads the number PC+x into PC. (PC<-PC+x). Recall that PC is already incremented when we start to execute the Call instruction.

Machine code format:

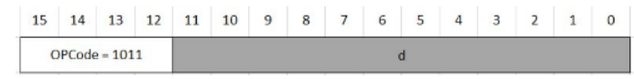


RET Instruction

Assembly format: Ret

Pops the top of the stack into PC.

Machine code format:



ONURCAN İŞLER😊

